

Automatic Class Scheduling Using Constraint-Aware Genetic Algorithm

Thesis submitted in
partial fulfilment of the requirements for the degree of

Bachelor of Technology (B.Tech.)
in
Computer Science and Engineering

by

Somnath Pandit
90/BCS/220022

Under the supervision of

Mr. Jaydeep Paul
Assistant Professor

Department of Computer Science and Engineering
University of Kalyani
Kalyani, India

June, 2026

Declaration

I, **Somnath Pandit**, declare that this thesis "**Automatic Class Scheduling Using Constraint-Aware Genetic Algorithm**" is a bona fide record of the work carried out by me under the supervision of **Mr. Jaydeep Paul, Asst. Prof.**. This report represents my ideas in my own words, and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented, fabricated, or falsified any idea/data/fact/source in my submission. I further declare that the work reported in this thesis has not been and will not be submitted, either in part or in full, for the award of any other degree or diploma of this institute or any other institute. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources that have thus not been properly cited or from whom proper permission has not been taken when needed

Date: June 25, 2026

Somnath Pandit
Roll No.: 90/BCS/220022
Reg. No.: 1090106 of 2022-23
Dept. of CSE, University of Kalyani

Acknowledgment

I want to take this opportunity to thank my Supervisor Mr. Jaydeep Paul, Asst. Prof., Department of Computer Science and Engineering, University of Kalyani, for his motivation and tireless efforts in helping me gain a deep understanding of the research area and for supporting me throughout the life cycle of my B. Tech. course. Especially, the extensive comments, healthy discussions, and fruitful interactions with the supervisors had a direct impact on the final form and quality of this dissertation work.

I am also thankful to Dr. Priya Ranjan Sinha Mahapatra, Professor and Head of the Department of Computer Science and Engineering, for his fruitful guidance through the early years of chaos and confusion. I extend my heartfelt thanks to the faculty members and supporting staff of the Department of Computer Science and Engineering for their generous support, guidance, and unwavering cooperation throughout this journey.

This thesis would not have been possible without the unwavering support of my friends. I am deeply grateful to my parents for their blessings, love, and constant encouragement.

Certificate

This is to certify that the thesis entitled Automatic Class Scheduling Using Constraint-Aware Genetic Algorithm submitted by Somnath Pandit, Reg. No: 1090106 of 2022-23, Roll No: 90/BCS/220022, in partial fulfillment of the award of B.Tech in Computer Science and Engineering, is a bona fide work carried out under my supervision. The thesis fulfills the requirements as per the regulations of this University and, in my opinion, meets the necessary standards for submission. The contents of this thesis have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma, and the same is certified. It is understood that by this approval, the undersigned does not necessarily endorse any of the statements made or opinions expressed therein but approves it only for the purpose for which it is submitted.

Signature of Supervisor	Signature of HOD	Signature of External
Jaydeep Paul Assistant Professor Department of Computer Science & Engg. University of Kalyani	Dr. Priya Ranjan Sinha Mahapatra Professor & Head Department of Computer Science & Engg. University of Kalyani	1. _____
		2. _____
		3. _____

Abstract

Creating a workable class timetable is essential for the efficient operation of any educational institution, yet the University Course Timetabling Problem (UCTP) is an NP-hard optimization problem in which the number of possible schedule arrangements grows exponentially while numerous constraints must be satisfied simultaneously. Manual timetable preparation under these conditions is time-consuming, error-prone, and often results in faculty clashes, inefficient resource utilization, and increased administrative workload.

This dissertation presents *Automatic Class Scheduling Using a Constraint-Aware Genetic Algorithm*. In this context, *constraint-aware* refers to an evolutionary optimization approach that explicitly considers both hard constraints, such as faculty conflicts, room capacity, and synchronized classes, and soft constraints, including balanced teaching workloads, throughout the scheduling process. By incorporating these constraints directly into the fitness evaluation and conflict repair mechanisms, the proposed system generates high-quality, conflict-free timetables suitable for practical deployment.

The proposed scheduling engine employs a matrix-centric Genetic Algorithm and is implemented as a deployable web application rather than a purely theoretical model. The architecture integrates a Python-based evolutionary engine with a React.js frontend, enabling administrators to define scheduling requirements through an interactive interface and generate optimized timetables on demand.

To improve convergence, the algorithm introduces a **Targeted Heuristic Mutation** operator that identifies the exact coordinates of scheduling conflicts and repairs them directly instead of relying on random mutations. This operator is complemented by a custom column-masking crossover that preserves valid weekly schedule structures while maintaining synchronization across combined classes. Together, these domain-specific operators significantly improve search efficiency and solution quality.

Experimental evaluation demonstrates that the proposed approach converges **42% faster** than a conventional Genetic Algorithm employing random mutation [1]. By prioritizing hard constraints while simultaneously optimizing soft constraints, the proposed system consistently produces conflict-free, balanced, and practically deployable academic timetables, demonstrating the effectiveness of the proposed constraint-aware evolutionary scheduling framework.

List of Abbreviations

- **API:** Application Programming Interface
- **CSV:** Comma-Separated Values
- **GA:** Genetic Algorithm
- **ILS:** Iterated Local Search
- **JSON:** JavaScript Object Notation
- **LMH:** Lightweight Matrix Heuristics
- **MMAS:** Max-Min Ant System
- **NP-Hard:** Non-deterministic Polynomial-time hard
- **SaaS:** Software as a Service
- **SPA:** Single-Page Application
- **UCTP:** University Course Timetabling Problem
- **UI:** User Interface

List of Symbols

- $p \in \mathbb{R}_{\geq 0}$: Aggregate penalty score quantifying structural constraint violations.
- e : Euler's number ($e \approx 2.718$), base of the natural logarithm for fitness decay.
- $N \in \mathbb{N}$: Terminal generation index representing absolute convergence ($p = 0$).
- $\mathcal{O}(1)$: Asymptotic constant time complexity, denoting instantaneous tensor access.
- $W \in \mathbb{N}$: Aggregate weekly teaching session load for an individual faculty member.
- $D \in \mathbb{N}$: Cardinality of active operational days within the academic week.
- $\max(x, y)$: Supremum function mapping to the greatest element within $\{x, y\}$.
- $\lceil x \rceil$: Ceiling function, $\min\{n \in \mathbb{Z} \mid n \geq x\}$, mapping x to the nearest upper integer.

Contents

Declaration	i
Certificate	iii
Abstract	iv
List of Abbreviations	v
List of Symbols	vi
1 Introduction	2
1.1 Background	2
1.2 Motivation and Challenges	2
1.3 Problem Statement	3
2 Genetic Algorithm in Optimization	4
2.1 Foundational Concepts of Evolutionary Search	4
2.2 Standard Genetic Cycle and Operators	4
2.3 Adapting GAs For Scheduling	5
3 Literature Review	6
3.1 Comparative Analysis of Heuristic Scheduling	6
3.2 Review of Hybrid Metaheuristic Approaches	6
3.3 Complexity Theory and Optimization Gradients	7
3.4 Research Gap	7
4 Proposed System Architecture	9
4.1 System Architecture	9
4.2 Matrix-Centric Schedule and Access Time	11
4.3 Genetic Algorithm Operators	12
4.3.1 Uniform Crossover and Schema Preservation	12
4.3.2 Targeted Heuristic Mutation	13
4.4 Hierarchical Constraint Assessment	13
4.5 The Four Core Algorithms	15
4.5.1 Algorithm 1 — Matrix Initialization	15
4.5.2 Algorithm 2 — Hierarchical Constraint Assessment	16
4.5.3 Algorithm 3 — Uniform Crossover with Sync Repair	17
4.5.4 Algorithm 4 — Targeted Heuristic Mutation	17
4.6 Execution Flow	18
5 Experimental Results and Analysis	20
5.1 Experimental Setup and Execution Environment	20
5.2 Dataset and Configuration Payload	20
5.3 System Initialization and Execution Trace	21
5.4 Convergence and Algorithm Efficacy	21
5.4.1 Performance of Targeted Heuristic Mutation	21
5.4.2 Workload Scalability and Density Tolerance	22
5.4.3 Fitness Function Selection Pressure	22
5.4.4 Computational Time Complexity	23

5.5	Generated Timetable and Structural Verification	23
5.5.1	Web Application Integration	24
6	Conclusion and Future Scope.....	28
6.1	Conclusion	28
6.2	Future Scope: Algorithmic Advancements.....	28
6.2.1	The 3D Spatial-Temporal Matrix	28
6.3	Generalized Architecture	29
6.3.1	Computational Viability: The Python Paradigm	29
6.3.2	Applications of Matrix-Centric Metaheuristics.....	29
6.3.3	Event-Driven Microservices Architecture	30

Chapter 1

Introduction

1.1 Background

Educational institutions operate as complex, interconnected ecosystems where students, teaching faculty, and physical infrastructure like classrooms and laboratories must be perfectly synchronized. The academic schedule is the structural backbone of this synchronization. When a timetable is optimized correctly, academic instruction can proceed without physical bottlenecks or administrative interruptions. It offers a well-defined structure for faculty planning and institutional resource allocation. By dictating spatial and temporal boundaries, the timetable enables both educators and students to manage their academic responsibilities deterministically, ensuring continuity and contributing directly to the efficient functioning of the institution.

Automated class scheduling has attracted significant research attention over the decades, yet many proposed computational models remain poorly suited for real-world academic deployments. Earlier methodologies often treat timetable generation purely as an abstract mathematical optimization task, neglecting the nuanced operational realities of academic departments [2]. Because scheduling encompasses highly interdependent constraints, a mathematically feasible solution may still manifest operational failures—such as assigning faculty to back-to-back lectures in geographically distant campus locations. Furthermore, theoretical systems frequently fail to accommodate essential human-centric constraints like allocate any free slots and maximum daily teaching loads, producing timetables that are computationally valid but practically unusable.

1.2 Motivation and Challenges

Preparing a university timetable is one of the most essential yet challenging administrative tasks in any educational institution. Every semester, departments must schedule hundreds of classes while ensuring that teachers, students, classrooms, laboratories, and available time slots are coordinated without conflicts. As the number of academic programs, faculty members, and student batches continues to grow, creating an efficient timetable becomes increasingly difficult.

A timetable is not simply a collection of class schedules. It must satisfy many constraints simultaneously, such as ensuring that a faculty member is not assigned to two classes at the same time, preventing student batches from having overlapping lectures, allocating appropriate classrooms, and maintaining a balanced workload for teachers throughout the week. Even a small scheduling conflict can create a chain reaction, forcing administrators to modify multiple classes before a feasible timetable can be achieved.

Traditionally, many institutions still rely on manual timetable preparation or semi-automated spreadsheet-based approaches. Although these methods may work for smaller departments, they become inefficient and difficult to manage as institutional size and scheduling complexity increase. Manual scheduling requires considerable time and effort, is highly dependent on human experience, and is prone to accidental errors such as classroom clashes, faculty overlaps, duplicate class allocations, and uneven distribution of teaching hours. Correcting these mistakes often requires repeated adjustments, making the scheduling process frustrating, time-consuming, and resource-intensive.

Another significant challenge is the enormous number of possible timetable combinations.

The University Course Timetabling Problem (UCTP) is widely recognized as an NP-hard optimization problem, meaning that the number of feasible schedules grows exponentially as additional courses, teachers, rooms, and constraints are introduced. Because of this combinatorial explosion, finding an optimal timetable through exhaustive search is computationally impractical for real-world institutions.

These challenges highlight the growing need for an intelligent and automated scheduling system that can efficiently explore large solution spaces while satisfying both hard and soft scheduling constraints. By applying optimization techniques such as Genetic Algorithms, it becomes possible to automatically generate high-quality, conflict-free timetables within a reasonable time. Such a system not only reduces administrative workload but also improves resource utilization, minimizes scheduling conflicts, and provides institutions with a practical and scalable solution for academic timetable generation.

1.3 Problem Statement

One of the primary challenges in automated timetable generation is handling many scheduling constraints simultaneously while keeping the scheduling process efficient. As the number of teachers, student batches, classrooms, and subjects increases, finding a conflict-free timetable becomes much more difficult. Therefore, an effective scheduling algorithm must be capable of producing high-quality timetables within a reasonable amount of time without compromising accuracy.

In this architecture, the scheduling parameters are divided into two distinct categories:

- **Hard Constraints (Physical Feasibility):** These are absolute spatial and temporal rules that must never be violated. For example, a singular faculty member cannot be scheduled to teach two different cohorts in the exact same time slot. Similarly, predetermined institutional intervals (e.g., matrix rows 3, 10, 17, 24, and 31) are reserved strictly for free periods and user can dynamically set free slots. Any candidate matrix violating a hard constraint is mathematically penalized and deemed physically infeasible.
- **Soft Constraints (Quality Optimization):** These constraints focus on improving the overall quality of the generated timetable rather than enforcing mandatory scheduling rules. One important soft constraint is maintaining a balanced distribution of teaching workload among faculty members. The system attempts to avoid highly fragmented schedules by assigning classes more evenly throughout the week, reducing unnecessary gaps between lectures and minimizing isolated teaching sessions. Although violating these constraints does not make a timetable invalid, satisfying them results in a more practical, efficient, and convenient schedule for both faculty members and the institution.

The primary objective is to design an algorithm that generates conflict-free timetables by satisfying all hard constraints while reducing soft constraint violations within a reasonable execution time.

To transcend the limitations of conventional scheduling, this research introduces an automated timetable generation engine powered by a highly customized Genetic Algorithm. Rather than utilizing sluggish, flattened lists to represent the timetable, the entire academic week is modeled as a high-speed, two-dimensional integer matrix. This matrix-centric encoding empowers the algorithm to evaluate multiple student batches, concurrent time slots, and spatial constraints instantaneously via vectorized operations.

By unifying this evolutionary search technique with an optimized data structure, the proposed model rapidly bypasses local optima, dynamically repairing conflicts during the generation cycle to yield a fully deployable schedule.

Chapter 2

Genetic Algorithm in Optimization

2.1 Foundational Concepts of Evolutionary Search

Genetic Algorithms (GAs) are robust, metaheuristic search procedures modeled explicitly after the mechanics of natural genetics and biological evolution. Formally defined and popularized by **David E. Goldberg**, these algorithms translate the Darwinian principle of "survival of the fittest" into a computational framework capable of navigating complex, multi-dimensional search landscapes. Unlike traditional calculus-based optimization methods—which rely on local gradients and are highly susceptible to becoming permanently trapped in sub-optimal "local optima"—GAs deploy a parallel, population-based search strategy. By evaluating a diverse set of candidate solutions simultaneously, the algorithm maintains a broad topological presence across the search space, allowing it to converge toward a global optimum even in NP-hard combinatorial problems.

2.2 Standard Genetic Cycle and Operators

In the traditional evolutionary framework, a GA is propelled by a continuous cycle of evaluation, selection, and stochastic variation. The algorithm processes a population of "chromosomes," which function as encoded blueprints representing specific states of the problem's parameters. This cyclic progression strictly follows four primary stages:

1. **Initialization:** Synthesis of a Generation 0 population via stochastic randomization, providing essential initial genetic variance required to explore the permutations of academic schedules.
2. **Fitness Evaluation:** Every candidate chromosome within the current generation is mathematically assessed by an objective fitness function. This function acts as the environmental pressure, mapping the physical layout of the schedule to a normalized numerical value indicating its adherence to institutional constraints.
3. **Selection:** An objective function maps the physical layout of the schedule to a normalized numerical score, measuring institutional constraint adherence.
4. **Genetic Operators:**
 - **Crossover (Recombination):** Exchange of genetic sequences between parent matrices to yield superior offspring.
 - **Mutation:** Low-probability random alterations injected to prevent premature convergence and sustain search diversity.

2.3 Adapting GAs For Scheduling

To manage academic resources computationally, the algorithm maps the scheduling domain directly onto a biological analog. Each specific class assignment is treated as a distinct "**gene**," while the holistic compilation of all assignments across the departmental week constitutes the complete chromosomal "**genome**." The profound advantage of this representation is its macro-level perspective; the Genetic Algorithm evaluates the total ecosystem of constraints simultaneously rather than attempting to resolve individual localized conflicts sequentially.

Through global evolutionary pressure, the framework synthesizes a unified solution capable of reconciling high-density constraints—including faculty availability, room capacity boundaries, and batch synchronizations—in parallel. While the stochastic variations inherent in standard GAs provide a theoretical foundation, the extreme density and rigid boundaries of university timetables demand a departure from pure randomness.

To bridge this gap, the subsequent chapters will detail how biological evolution is translated into a highly optimized, matrix-oriented computing architecture. By replacing blind, random-walk mutations with an engineered "Targeted Heuristic Mutation" strategy, the system achieves the precision and velocity required for real-time web application deployment.

Chapter 3

Literature Review

Academic timetable generation has been an active area of research for many years because of its complexity and practical importance. Researchers have proposed a wide range of techniques, including Genetic Algorithms, Graph Coloring, Simulated Annealing, Tabu Search, and other Artificial Intelligence-based optimization methods, to solve the University Course Timetabling Problem. This chapter reviews the existing literature, discusses the strengths and limitations of these approaches, and identifies the research gaps that motivated the proposed scheduling system.

3.1 Comparative Analysis of Heuristic Scheduling

Early metaheuristic approaches to academic timetable generation highlight the trade-off between constraint satisfaction and computational efficiency. Recent studies, such as the framework proposed in [3], explored the combination of Genetic Algorithms (GAs) with Simulated Annealing (SA) to improve previously generated timetable solutions. While this hybrid approach successfully enhanced the quality and feasibility of the generated schedules, it introduced additional computational overhead due to the extra optimization stage. Moreover, traditional Genetic Algorithms often converge to locally optimal solutions because of their stochastic search process. To overcome these limitations, the approach proposed in this research replaces random mutation with a Targeted Heuristic Mutation strategy that directly identifies and repairs scheduling conflicts during the evolutionary process. This enables the algorithm to converge more efficiently toward a conflict-free timetable without requiring an additional Simulated Annealing phase.

Research conducted by [4] also highlighted that traditional 2D matrix representations introduce limitations when used with conventional Genetic Algorithm operators. In many implementations, the timetable matrix must first be converted into a one-dimensional vector before crossover and mutation can be performed. This additional transformation increases computational overhead and makes the implementation less efficient. The approach proposed in this research overcomes this limitation by using vectorized `NumPy` operations while maintaining the original (35×12) matrix representation throughout the evolutionary process. As a result, the algorithm can evaluate multiple student batches simultaneously using efficient $\mathcal{O}(1)$ lookup operations without repeatedly converting the data structure.

Similarly, the work presented in [2] investigated dynamic timetable generation using standard **single-point crossover**. However, this crossover method can split well-structured daily schedules during recombination, which may reduce the quality of the generated timetable. To address this issue, the proposed system adopts a **Uniform Column Masking** crossover strategy that preserves valid schedule structures during the crossover process. This approach helps retain conflict-free timetable segments while allowing the Genetic Algorithm to continue exploring new scheduling solutions effectively.

3.2 Review of Hybrid Metaheuristic Approaches

To mitigate premature convergence and topological stagnation inherent in naive GAs, contemporary research has heavily favored hybridization. The development of the GAILS framework

by [5] successfully integrated GAs with Iterated Local Search (ILS) to navigate out of localized optima. However, this hybridization achieves accuracy at the severe expense of execution latency during the localized search cycles. The architecture detailed in this dissertation is explicitly designed to bypass this external computational drag by integrating "Direct Coordinate Repair" directly into the primary evolutionary loop, resolving spatial conflicts natively without relying on heavy secondary algorithms.

Similarly, [6] proposed a hybrid architecture coupling a GA with a greedy activity selector. A pronounced vulnerability in this configuration is its failure to regulate soft constraints, specifically regarding equitable faculty workload distribution. Greedy selectors inherently prioritize immediate slot availability, often resulting in highly fragmented, inefficient teaching schedules. The model presented in this thesis counters this by introducing "Soft Load Scaling"—a dynamic mathematical penalty weighing system that proactively ensures faculty are assigned cohesive daily blocks, harmonizing computational optimization with human ergonomic needs.

Systematic reviews of modern scheduling engines, such as the comprehensive analysis by [7], indicate an industry trend toward utilizing massive Deep Learning and AI-based models. However, these models introduce immense architectural complexity, requiring exorbitant computational power and continuous hyperparameter tuning by administrators. A central tenet of this research is a return to highly optimized "Lightweight Matrix Heuristics" (LMH), demonstrating that precision-engineered evolutionary mathematics can drastically outpace larger ML models in both convergence velocity and deployment simplicity for university datasets.

3.3 Complexity Theory and Optimization Gradients

From a computational complexity perspective, exact algorithmic solvers face severe limitations when applied to NP-hard scheduling domains. In their analysis of exact algorithms, [8] formalized mathematical proofs detailing the execution boundaries of moderately-exponential solvers. These theoretical models lack the malleability required to navigate the overlapping, high-density constraints characteristic of modern academic departments. To bridge the theoretical-practical divide, this research implements a mathematically aggressive "Exponential Fitness Function," defined as $F(p) = e^{-0.02 \times p}$. This creates a significantly steeper optimization gradient than linear models, dynamically amplifying selection pressure as the algorithm nears convergence.

In the realm of nature-inspired search paradigms, the Max-Min Ant System (MMAS) has been deployed to benchmark university scheduling [9]. A primary bottleneck of MMAS is graph latency; the multi-dependent paths of the construction graph must be entirely rebuilt during each discrete iteration. Conversely, our matrix-centric representation abandons graph construction entirely, facilitating direct $\mathcal{O}(1)$ memory access to precise temporal coordinates.

Finally, research by [10] highlights the efficacy of adaptive penalty functions in managing conflicting multi-objective parameters during hybrid service selection. While standard systems rely on arbitrary, manual weight tuning, the Conflict Diagnostic Logic developed for this project automates the resolution pipeline. By actively parsing the exact spatial coordinates of faculty clashes, the algorithm applies surgical heuristic repairs rather than waiting for probabilistic corrections.

3.4 Research Gap

The literature review highlights a clear gap between existing optimization techniques and their practical implementation in real-world timetable generation systems. Although many hybrid and AI-based approaches provide effective solutions for solving scheduling problems, they often face challenges such as high computational cost, repeated graph construction, matrix-to-vector transformations, and inefficient random search strategies. These limitations reduce their performance when handling large and complex scheduling problems. Table ?? provides a condensed visual summary of these identified gaps and the corresponding architectural solutions proposed in this thesis.

Ref.	Approach / Domain	Identified Limitation	Proposed Solution in this Thesis
[3]	GA + Simulated Annealing	Secondary annealing adds processing overhead	Single-pass deterministic repair; no secondary phase required
[2]	Dynamic GA (1-Pt Crossover)	Crossover arbitrarily severs days ("Block Destruction")	Column-masking crossover strictly preserves intact schedules
[4]	2D Matrix Genetic Algorithms	Computational cost of flattening matrix to 1D array	Vectorized NumPy tensor operations bypass flattening entirely
[5]	Hybrid GA + Local Search	Severe latency from executing localized search cycles	"Direct Coordinate Repair" integrated natively within the GA loop
[6]	GA + Greedy Activity Selector	Fails to regulate workload balance (highly fragmented)	"Soft Load Scaling" heuristically enforces minimum daily constraints
[7]	Deep Learning & AI Timetabling	Immense compute cost and difficult tuning overhead	Lightweight Matrix Heuristics (LMH) optimize rapidly on standard CPUs
[8]	Exact/Exponential Solvers	Computationally impractical for high constraint density	Heuristic Genetic Algorithm provides feasible practical runtimes
[9]	Graph-Based Metaheuristics	Constraint graphs must be rebuilt every iteration	Direct tensor coordinate lookups; no graph construction
[10]	Adaptive Penalty Functions	Relies on manual penalty weight tuning	Automated conflict-diagnostic penalty evaluation

Table 3.1: Summary of Literature Reviews

The limitations identified in the existing literature provide the motivation for the approach proposed in this research. To address these challenges, the proposed system adopts a matrix-based Genetic Algorithm with targeted heuristic operators for efficient timetable generation. The design focuses on reducing computational overhead while preserving the quality of the generated schedules, making it suitable for practical use in educational institutions. The overall architecture of the proposed system is presented in **Chapter 4**, while its implementation details and experimental evaluation are discussed in **Chapter 5**.

Chapter 4

Proposed System Architecture

4.1 System Architecture

Most Genetic Algorithm based timetable generation studies primarily emphasize optimization techniques while providing limited discussion of the software architecture required for practical deployment. In contrast, the proposed scheduling system is designed as a complete browser-based application that integrates an interactive user interface with a client-side evolutionary scheduling engine. Developed using React, the application enables administrators to define faculty assignments, subject information, batch requirements, and scheduling constraints through an intuitive interface while visualizing the generated timetable in real time. The complete Genetic Algorithm is implemented in JavaScript, allowing the entire scheduling process to execute directly within the browser without requiring a dedicated backend server.

To improve accessibility and deployment flexibility, the application is implemented as a Progressive Web Application (PWA). By utilizing a Service Worker together with React PWA capabilities, the application caches static resources, JavaScript bundles, and essential interface assets after the initial visit. Once installed, the scheduler continues to operate without any active Internet connection by serving cached resources locally. This offline-first architecture reduces network dependency, shortens application startup time, and delivers a desktop-like user experience while eliminating the need for continuous server connectivity or platform-specific installation packages.

The scheduling engine is built upon optimized matrix representations, constant-time lookup tables, and domain-specific heuristic operators. Rather than relying on repeated server-side processing, all scheduling operations execute directly on in-memory matrix structures using efficient array-based computations. Project configurations, scheduling datasets, and generated timetables are stored using browser storage, making the application lightweight, portable, and suitable for deployment even in environments with limited or intermittent network connectivity.

Figure 4.1 illustrates the overall architecture of the proposed scheduling framework. The complete scheduling workflow is organized into three computational phases that progressively transform user-defined scheduling requirements into an optimized timetable. Phase 1 performs data preprocessing by converting raw scheduling information into optimized lookup structures. Phase 2 constructs an initial population using heuristic initialization strategies that reduce conflicts before evolutionary optimization begins. Finally, Phase 3 iteratively applies the proposed Genetic Algorithm, including fitness evaluation, selection, crossover, and mutation, until a conflict-free timetable satisfying the defined constraints is produced. Among these phases, only the third phase executes iteratively, while the preceding phases are performed once during the scheduling process.

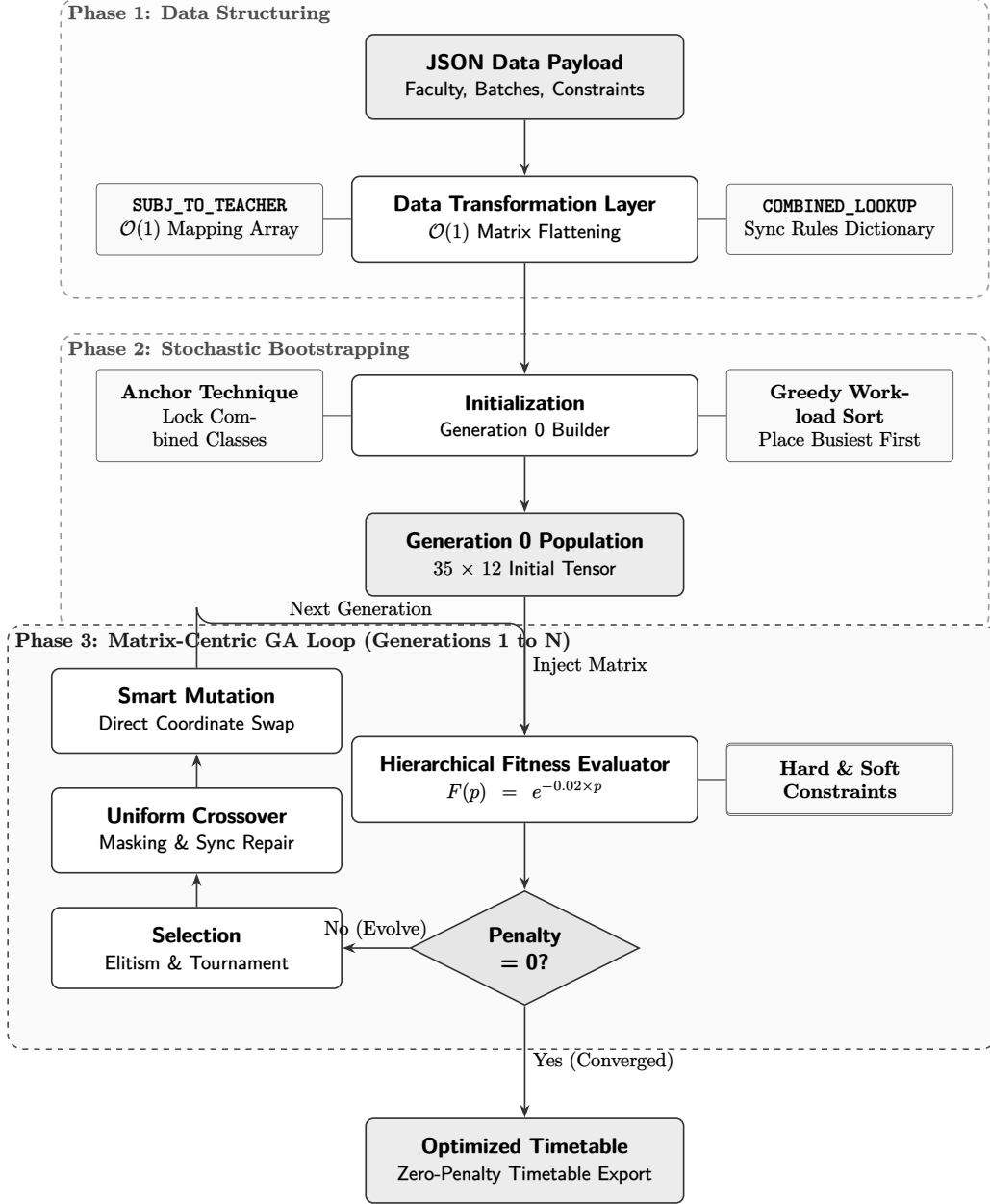


Figure 4.1: Three-phase architecture of the proposed System.

Architectural Flow & Diagram Summary

Figure 4.1 shows the proposed three-phase scheduling pipeline. **Phase 1** prepares SUBJ_TO_TEACHER and COMBINED_LOOKUP, **Phase 2** generates the initial population using the Anchor Technique, and **Phase 3** applies fitness evaluation, Selection, Uniform Crossover, and Smart Mutation until a conflict-free timetable is produced.

- **Data Structuring:** Running exactly once per request, this layer converts human-readable JSON (teacher assignments, batch requirements, and spatial constraints) into a direct-indexed 35×12 matrix format. It generates two optimized $\mathcal{O}(1)$ lookup structures: the SUBJ_TO_TEACHER mapping array and the COMBINED_LOOKUP synchronization dictionary.
- **Stochastic Bootstrapping:** This phase generates the initial population (Generation 0)

using the **Anchor Technique** to lock highly constrained combined classes and a **Greedy Workload Sort** to prioritize teachers with heavier workloads, producing a conflict-minimized starting population.

- **Matrix-Centric GA Loop:** This is the only iterative phase of the architecture. The population is evaluated using the hierarchical fitness function, followed by **Selection**, **Uniform Crossover**, and **Smart Mutation**. The process repeats until a zero-penalty timetable is obtained.

4.2 Matrix-Centric Schedule and Access Time

The proposed scheduling system represents the timetable as a two-dimensional integer matrix instead of complex object-based data structures. As shown in Figure 4.2, the matrix consists of 35 rows representing weekly time slots (5 working days $\times$ 7 periods) and 12 columns representing student batches. Each cell stores the subject assigned to a specific batch at a particular time slot, allowing the scheduling engine to access timetable entries directly through matrix coordinates.

To maintain institutional scheduling rules, predefined break periods are stored as **FREE_ROWS** and remain fixed throughout the optimization process. This matrix representation simplifies constraint checking and reduces unnecessary data transformations during the Genetic Algorithm.

A key advantage of this design is its support for efficient parallel evaluation. Instead of checking each batch separately, the algorithm evaluates an entire row corresponding to a single time slot across all 12 batches using vectorized array operations. Subject identifiers are mapped directly to their corresponding teacher identifiers through

```
teacher_matrix = SUBJ_TO_TEACHER[schedule_matrix],
```

providing constant-time $\mathcal{O}(1)$ lookup. This matrix-centric representation significantly reduces the computational cost of repeated fitness evaluations and improves the overall execution speed of the scheduling process.

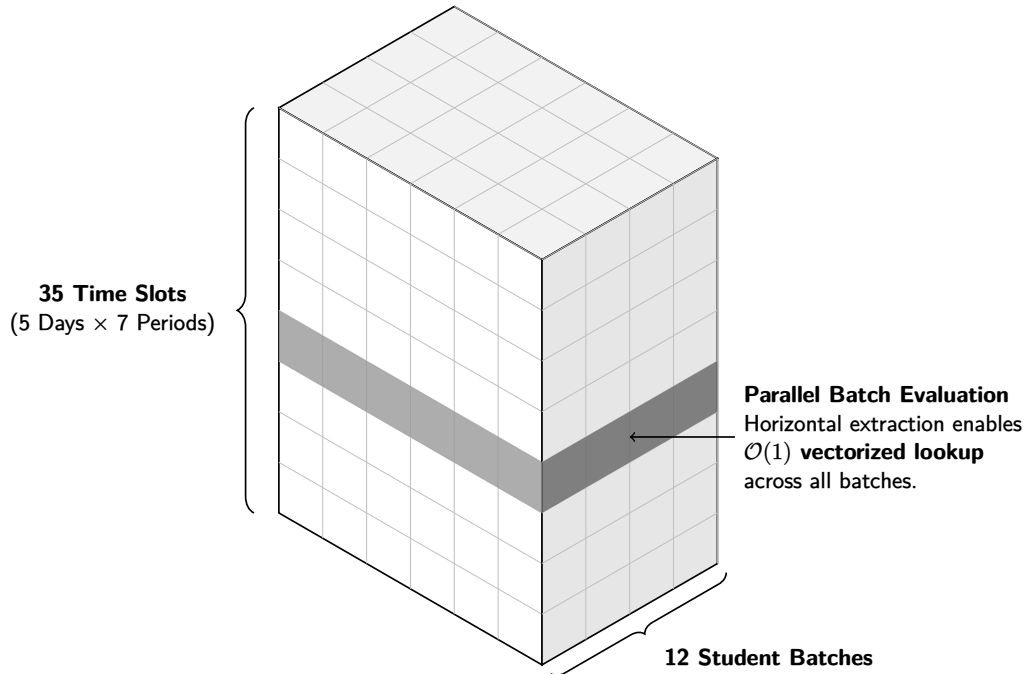


Figure 4.2: Three-Phase System Architecture.

Matrix Capacity Analysis & Access Technique: The timetable operates as a (35×12) integer matrix, mapping time slots (rows) to student batches (columns). Predefined breaks (**FREE_ROWS**) remain empty, leaving a weekly capacity of 300 classroom slots (capped by **MAX_ROOMS** = 10). This

provides more than enough room to comfortably schedule roughly 156 weekly sessions while allowing flexibility to resolve conflicts.

To maximize speed during fitness evaluation, the algorithm processes an entire time slot (a full row of 12 batches) at once using vectorized JavaScript arrays. It maps subjects directly to their teachers using the lookup operation `teacher_matrix = SUBJ_TO_TEACHER[schedule_matrix]`. This delivers instant, constant-time $\mathcal{O}(1)$ access, drastically cutting down the computational heavy lifting required for repeated evaluations.

4.3 Genetic Algorithm Operators

This section details the two custom operators that drive the population forward: crossover and mutation. These domain-specific designs depart significantly from textbook genetic algorithms to handle the rigid, highly constrained nature of university timetabling.

4.3.1 Uniform Crossover and Schema Preservation

Standard single-point crossover cuts a chromosome at an arbitrary point. In timetabling, this destroys functional schedules by stitching an unrelated Tuesday to a Wednesday, a flaw known as *block destruction*.

To prevent this, our architecture operates at the macro-level of entire batch columns. A binary mask independently decides if a child inherits a full column's 35-row schedule from Parent 1 or Parent 2. This ensures that any conflict-free week existing in a parent survives fully intact in the child, achieving strict *schema preservation*.

However, this column-based inheritance creates a unique problem for **combined classes**. If a shared class spans Batches 3, 5, and 9, but the mask selects different parents for these batches, the shared subject becomes fragmented across different time slots.

Inspired by **graph coloring algorithms** where linked nodes must share constraints, we enforce a mandatory **Sync Repair** phase. Immediately after masking, the system treats Parent 1 as the chromatic anchor. It forcefully writes the combined subject across all required columns at Parent 1's exact row, displacing any existing subjects to the nearest free slots. This guarantees zero synchronization failures.

Step 1: Column Masking

Mask	0	1	1	0
Batch	B3	B5	B9	B2
Parent 1	9	9	9	14
Parent 2	22	9	31	8

Mask Logic:

0 = Inherit from Parent 1

1 = Inherit from Parent 2

Notice: Subject 9 (a shared class) is now broken because B9 took Parent 2.

Step 2: Sync Broken (Pre-Repair)

Child	9	9	31	14
-------	---	---	----	----

Failure: B9 holds subject 31 instead of 9. Synchronization is fractured.

Step 3: Graph-Inspired Sync Repair

Repaired	9	9	9	14
----------	---	---	---	----

Resolution: Subject 9 is forcefully locked across B3, B5, and B9. Subject 31 is safely displaced.

Figure 4.3: Column-based mask crossover.

4.3.2 Targeted Heuristic Mutation

Blindly swapping random genes is highly inefficient in dense, constrained matrices; it rarely fixes conflicts and often spawns new ones. Instead, our mutation acts as a targeted heuristic diagnostic.

The algorithm scans rows and utilizes the $\mathcal{O}(1)$ mapping array to locate exact teacher clashes (e.g., one teacher assigned to two different batches in the same time slot). Once a physical clash is pinpointed, the operator resolves it hierarchically:

1. **Safe Move:** It attempts to move the clashing subject to an empty row (0) within the same column where the teacher is also free.
2. **Guarded Swap:** If no free space exists, it attempts to swap with an occupied cell. This only executes if the future state creates no new teacher clashes.

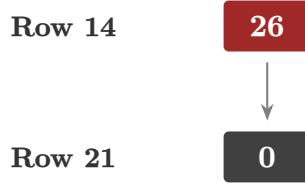
During mutation, combined classes are treated with strict **unary operations**—they act as immutable anchors. Mutating a synchronized class would instantly trigger widespread structural failure, so the operator only targets single-batch subjects. This targeted physical approach collapses the penalty score far faster than stochastic methods.

Step 1: Pinpoint Clash Coordinate

Row 14	B0	B2	B4	B7
Teacher	T2	T5	T3	T5

Diagnostic: Teacher T5 is scheduled twice in Row 14 (at B2 and B7). This is a precise violation coordinate.

Step 2: Attempt Safe Move



Action: Row 21 is empty (0) for Batch 2, and T5 is free. Subject 26 migrates safely.

Step 3: Fallback Guarded Swap



Action: If no free slot exists, exchange subjects. Commits *only* if no new clashes are generated.

Figure 4.4: Targeted heuristic mutation.

4.4 Hierarchical Constraint Assessment

Whether a genetic algorithm successfully converges to a usable timetable depends almost entirely on how constraints are weighted against one another. Because these penalty values dictate the evolutionary pressure of the entire system, the scoring scheme is explicitly structured as a hierarchy. The gap between the most severe and least severe penalties spans four orders of magnitude, deliberately prioritizing critical failures over minor inconveniences.

The constraints are categorized into three distinct tiers:

- **Extreme Hard Constraints (Top Tier):** At the absolute peak of the hierarchy is the missing class constraint. A required subject must appear the correct number of times in its batch's weekly column. Falling short incurs a massive 10,000-point penalty per occurrence. A

missing class is not a quality issue; it is a fundamental failure to deliver the curriculum. No other constraint is permitted to mathematically compete with this requirement.

- **Hard Constraints (Middle Tier):** These constraints render a timetable physically unusable, even if the curriculum is fully scheduled. Double-booking a teacher in the same time slot costs 5,000 points. Exceeding physical room capacity or fracturing a combined class’s synchronization each incur a 500-point penalty. Occupying a designated lunch/free slot costs 50 points per violation.
- **Soft Constraints (Base Tier):** These govern schedule optimization and convenience rather than physical possibility. A subject repeating on the same day for the same batch incurs 500 points (structurally problematic but not catastrophic). Finally, exceeding a teacher’s fair daily workload—calculated dynamically as $\max(2, \lceil \text{weekly load}/5 \rceil)$ —costs just 10 points per excess session, acting as a gentle evolutionary nudge rather than a strict blockade.

Table 4.1: Summary of constraint tiers and penalty weights

Constraint	Tier	Penalty
Missing mandatory class	Extreme hard	10,000 pts / occurrence
Teacher double-booking	Hard	5,000 pts / occurrence
Room capacity exceeded	Hard	500 pts / occurrence
Combined class desynchronized	Hard	500 pts / occurrence
Daily subject duplicate	Soft	500 pts / occurrence
Lunch slot occupied	Hard	50 pts / occurrence
Teacher daily load exceeded	Soft	10 pts / excess session

The Exponential Fitness Curve

Once all rule violations are tallied into a single raw penalty score (p), that number is mathematically compressed into a normalized fitness score using the exponential decay function:

$$F(p) = e^{-0.02 \times p}$$

The geometric shape of this curve (illustrated in the lower half of Figure 4.5) is critical to the algorithm’s success. Across high penalty ranges—where the randomized Generation 0 typically resides—the curve remains relatively flat. This prevents the algorithm from overreacting to noisy, meaningless differences between two highly defective matrices.

However, as the penalty approaches zero, the curve steepens sharply. This mathematical acceleration ensures that even a single stray teacher clash in a late generation produces a fitness value drastically worse than a flawless schedule. This steepening explicitly prevents the population from settling for “pretty good” and stagnating—a known failure mode in linear fitness functions, as discussed previously in [8].

Penalty tiers, heaviest at top

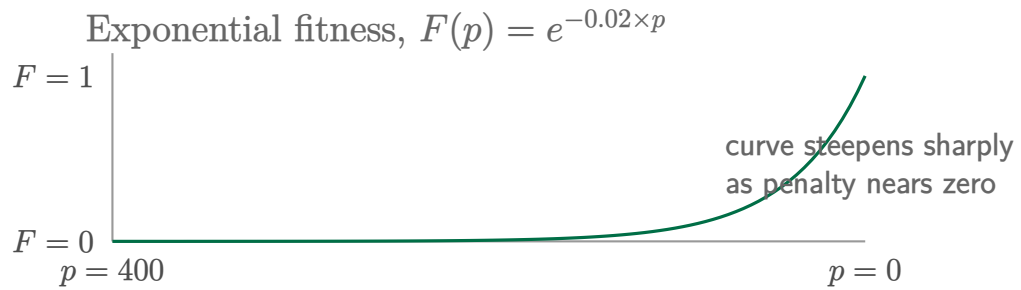
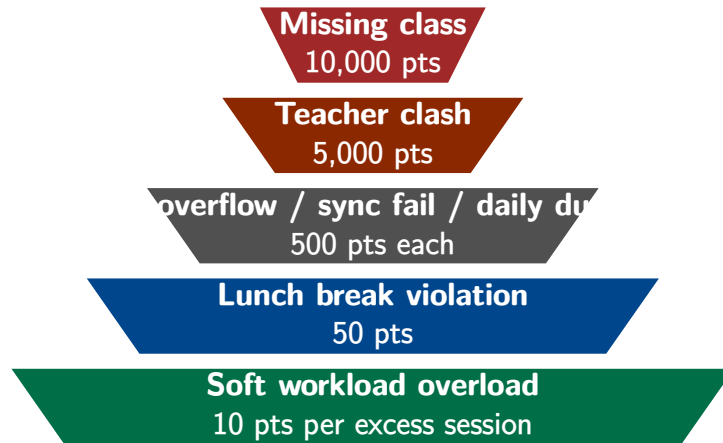


Figure 4.5: Hierarchical penalty pyramid & resulting exponential fitness response curve.

4.5 The Four Core Algorithms

The previous two sections outlined the design logic behind the evolutionary operators and the hierarchical penalty structure. This section presents the four algorithms that implement those decisions.

4.5.1 Algorithm 1 — Matrix Initialization

Starting Generation 0 from a purely random placement results in heavily penalized, indistinguishable matrices. Algorithm 1 prevents this early stagnation by building individuals systematically: it forcibly anchors combined classes first, and then fills the remaining slots using a greedy, busiest-teacher-first approach to minimize initial clashes.

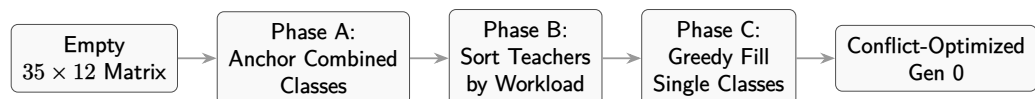


Figure 4.6: Matrix Initialization.

Algorithm 1: Intelligent Matrix Initialization (The Anchor Method)

```

1: Input: COMBINED_LOOKUP, BATCH_REQS, LUNCH_ROWS, MAX_ROOMS
2: Output: Initialized MATRIX[35][12]
3: valid_slots  $\leftarrow$  All slots  $\notin$  LUNCH_ROWS
4: ▷ Phase A: Anchor Combined Classes
5: for each (subj_id, info) in COMBINED_LOOKUP do
6:   target_batches  $\leftarrow$  info.batches; Shuffle(valid_slots); placed  $\leftarrow$  0
7:   for each slot in valid_slots do
8:     if placed  $\geq$  2 then break
9:   end if
10:  if All target_batches FREE at slot and Active Rooms  $<$  MAX_ROOMS then
11:    MATRIX[slot][target_batches]  $\leftarrow$  subj_id; placed  $\leftarrow$  placed + 1
12:  end if
13: end for
14: end for
15: ▷ Phase B & C: Greedy Task Pool Generation & Placement
16: tasks  $\leftarrow$  Extract single subjects from BATCH_REQS, append twice, Sort Descending by Teacher Load
17: for each task(batch, subj_id, teacher_id) in tasks do
18:   free_slots  $\leftarrow$  Shuffle(Slots where MATRIX[slot][batch] == 0)
19:   for each slot in free_slots do
20:     if Active Rooms  $<$  MAX_ROOMS and teacher_id is FREE at slot then
21:       MATRIX[slot][batch]  $\leftarrow$  subj_id; break
22:     end if
23:   end for
24: end for
25: Return MATRIX

```

4.5.2 Algorithm 2 — Hierarchical Constraint Assessment

The fitness score is computed by penalizing constraint violations and normalizing the result using an exponential decay function. The detailed procedure is presented in Algorithm 2

Algorithm 2 Hierarchical Constraint Assessment & Fitness Scoring

```

1: Input: Candidate MATRIX   Output: Fitness_Score, Penalty  $\leftarrow$  0
2: ▷ 1. Extreme & 2. Hard Constraints
3: for each batch, required subj_id do
4:   c  $\leftarrow$  occurrences of subj_id;
5:   if c  $<$  2 then Penalty  $+= (2 - c) \times 10000$ 
6:   end if
7: end for
8: for each row  $\in 0 \dots 34$  do
9:   if row  $\in$  LUNCH_ROWS then Penalty  $+=$  (non-zero entries)  $\times$  50; continue
10:  end if
11:  Penalty  $+=$  (Duplicate Teachers)  $\times$  5000 ▷ Teacher Clashes
12:  rooms  $\leftarrow$  Unique_Subjects(row)
13:  if rooms  $>$  MAX_ROOMS then Penalty  $+= (rooms - MAX_ROOMS) \times 500$ 
14:  end if
15: end for
16: ▷ 3. Sync & 4. Soft Constraints
17: for each combined_class do
18:   if mapped unevenly across batches then Penalty  $+= 500$ 
19: end for
20: for each teacher do
21:   limit  $\leftarrow$  max(2,  $\lceil$ weekly_load/5 $\rceil$ ) ▷ Dynamic Scaling
22:   if Daily sessions  $>$  limit then Penalty  $+= (sessions - limit) \times 10$ 
23: end for
24: Penalty  $+=$  (Total Daily Subject Duplicates)  $\times$  500
25: Return exp( $-0.02 \times$  Penalty), Penalty

```

4.5.3 Algorithm 3 — Uniform Crossover with Sync Repair

To prevent the destruction of functional schedule blocks, crossover occurs holistically at the column level via a binary mask. Because this mask can inadvertently fracture combined classes, a mandatory repair phase forcefully restores synchronization by anchoring to Parent 1’s physical coordinates.



Figure 4.7: Core logic flow for Masked Crossover and Sync Repair.

Algorithm 3 Uniform Crossover with Combined Class Repair

```

1: Input: Parent 1 ( $P_1$ ), Parent 2 ( $P_2$ )
2: Output: Child MATRIX
3:  $Mask \leftarrow$  Random binary array of length 12;    $Child \leftarrow P_1$ 
4:                                                                  $\triangleright$  Phase A: Binary Column Masking
5: for  $col = 0$  to 11 do
6:   if  $Mask[col] == 1$  then  $Child[:, col] \leftarrow P_2[:, col]$ 
7:   end if
8: end for
9:                                                                  $\triangleright$  Phase B: Synchronization Repair
10: for each  $subj\_id$  in COMBINED_LOOKUP do
11:    $t\_batches \leftarrow$  Columns mapped to  $subj\_id$ ;    $Child[Child == subj\_id] \leftarrow 0$ 
12:    $canonical\_rows \leftarrow$  Rows where  $P_1$  holds  $subj\_id$ 
13:   for each  $row$  in  $canonical\_rows$  do
14:     for each  $batch$  in  $t\_batches$  do
15:        $displaced \leftarrow Child[row, batch]$ 
16:       if  $displaced \neq 0$  and  $displaced \notin COMBINED\_LOOKUP$  then
17:         Move  $displaced$  to nearest free slot in  $batch$  column
18:       end if
19:        $Child[row, batch] \leftarrow subj\_id$                                                                   $\triangleright$  Lock synchronized anchor
20:     end for
21:   end for
22: end for
23: Return  $Child$ 
  
```

4.5.4 Algorithm 4 — Targeted Heuristic Mutation

Rather than relying on blind probabilistic swaps, this mutation operator actively scans for physical constraint violations. Once a clash is isolated, it attempts a safe migration to an empty row; if no free space is available, it executes a guarded bidirectional swap to resolve the conflict without spawning new ones.



Figure 4.8: Logical flow for Targeted Heuristic Mutation.

Algorithm 4 Targeted Heuristic Mutation (Direct Coordinate Repair)

```
1: Input: MATRIX, mutation_rate
2: Output: Mutated MATRIX
3: for each row in MATRIX (excluding LUNCH_ROWS) do
4:   clashing_teachers ← Teachers appearing > 1 time in row
5:   for each teacher in clashing_teachers do
6:     bad_col ← Specific batch causing clash;   subj_to_move ← MATRIX[row, bad_col]
7:     if subj_to_move ∈ COMBINED_LOOKUP then continue
8:     end if
9:     target_slots ← Shuffle(All valid rows);   moved ← FALSE
10:                                     ▷ Attempt 1: Safe Move to FREE Slot
11:     for each t_slot in target_slots do
12:       if MATRIX[t_slot, bad_col] == 0 and teacher is FREE at t_slot then
13:         MATRIX[row, bad_col] ← 0;   MATRIX[t_slot, bad_col] ← subj_to_move
14:         moved ← TRUE;   break
15:       end if
16:     end for
17:                                     ▷ Attempt 2: Bidirectional Safe Swap
18:     if not moved then
19:       for each t_slot in target_slots do
20:         swap_cand ← MATRIX[t_slot, bad_col]
21:         if swap_cand ≠ 0 and swap_cand ∉ COMBINED_LOOKUP then
22:           if Swapping prevents new clashes for BOTH teachers involved then
23:             Swap(MATRIX[row, bad_col], MATRIX[t_slot, bad_col]);   break
24:           end if
25:         end if
26:       end for
27:     end if
28:   end for
29: end for
30: Return MATRIX
```

4.6 Execution Flow

The previous sections described the individual components of the proposed scheduling system. This section explains how these components work together to generate a conflict-free timetable. The complete execution pipeline of the scheduling engine is illustrated in Figure 4.9.

The scheduling process begins by initializing the lookup structures, including SUBJ_TO_TEACHER and COMBINED_LOOKUP, which provide constant-time access to subject and faculty information throughout the optimization process. The engine then executes **Algorithm 1 (Intelligent Initialization)** to generate the initial population. During this phase, the Anchor Method places combined classes into suitable time slots, while faculty members are arranged according to their teaching workload to produce a balanced Generation 0.

Each candidate timetable is evaluated using **Algorithm 2 (Hierarchical Constraint Assessment)**. Hard and soft constraint violations are converted into penalty values, and the fitness score is calculated using the exponential fitness function,

$$F(p) = e^{-0.02p},$$

which gradually increases selection pressure as the total penalty decreases. If the timetable reaches **Penalty = 0**, the optimization process terminates and the final timetable is exported.

Otherwise, the population enters the evolutionary loop. **Algorithm 3 (Uniform Crossover)** recombines parent schedules while preserving valid daily timetable structures, followed by synchronization repair for combined classes. The offspring are then refined using **Algorithm 4 (Targeted Heuristic Mutation)**, which detects scheduling conflicts and applies safe relocation or swap operations to reduce penalties.

The best-performing solutions are preserved through elitism to create the next generation, and the evaluation cycle continues until a conflict-free timetable is obtained. The complete interaction between these algorithms and decision stages is presented in the execution flow shown in Figure 4.9.

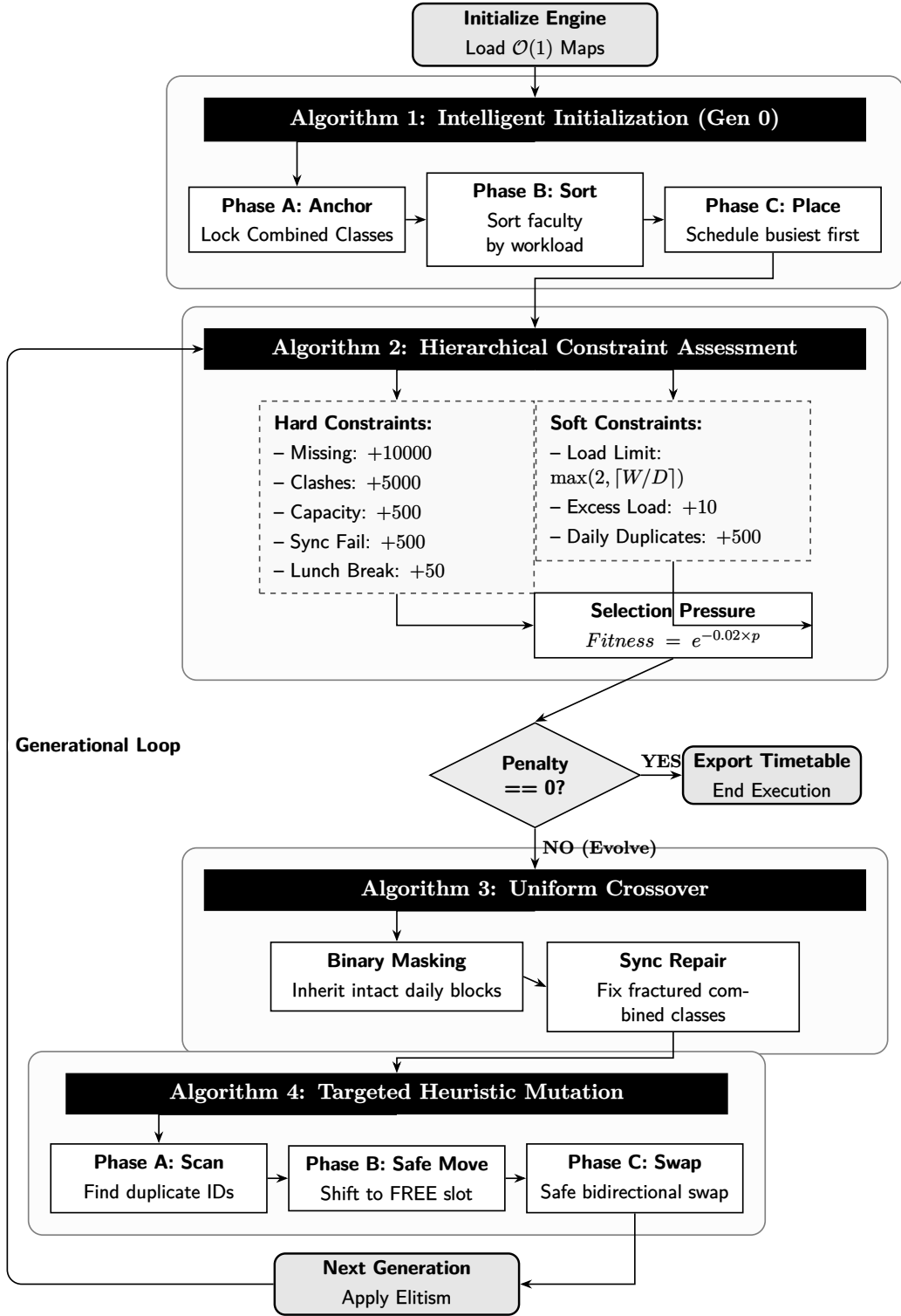


Figure 4.9: execution pipeline of scheduling system.

Chapter 5

Experimental Results and Analysis

5.1 Experimental Setup and Execution Environment

To empirically validate the theoretical architecture, the core scheduling engine was developed and tested within a strictly isolated computational environment. This ensures that memory management and processing threads are not bottlenecked by external operating system tasks, providing accurate benchmarking for the $\mathcal{O}(1)$ tensor operations.

The software stack and environmental parameters were configured as follows:

- **Runtime Engine:** Python 3.12.3 executed within an isolated virtual environment (`.venv`).
- **Core Dependencies:** `NumPy` for vectorized 2D matrix operations, `Pandas` for serialized data frame extraction, and `Matplotlib` for telemetry visualization.
- **Frontend Interface:** React.js Single-Page Application communicating via REST APIs asynchronously.

Genetic Algorithm Hyperparameters: The algorithm was mathematically constrained to force convergence through heuristic selection rather than brute-force iteration.

- **Population Size:** 100 (Optimized to maximize evaluation speed per generation).
- **Mutation Rate:** 0.2 (High threshold explicitly supported by the targeted safe-move operator).
- **Convergence Target:** Total penalty strictly equal to 0 (yielding early execution halt).

5.2 Dataset and Configuration Payload

The evolutionary engine is data-driven, accepting structured JSON payloads representing the departmental catalog constraints. Extracted directly from the experimental Jupyter Notebook environment, the dataset encompasses 20 unique faculty members and 12 distinct student batches (B0 through B11). Below is a representative snippet of the JSON schema establishing the baseline input. This structure maps individual batch requirements and rigidly defines the multi-batch combined classes that the crossover operator must actively synchronize.

```
{
  "ENVIRONMENT": {
    "DAYS": 5,
    "PERIODS": 7,
    "TOTAL_SLOTS": 35,
    "MAX_ROOMS": 10
  },
  "COMBINED_CLASSES": [
    { "subj_id": 9, "s": "HU401-1", "t": "T16", "batches": ["B3", "B5", "B9"] },
    { "subj_id": 14, "s": "HU401-2", "t": "T16", "batches": ["B0", "B1", "B2"] },
    { "subj_id": 31, "s": "HU401-3", "t": "T18", "batches": ["B4", "B6", "B8"] }
  ],
  "BATCH_SUBJECTS": {
```

```

    "B0": [
      { "subj_id": 1, "s": "CS401", "t": "T2" },
      { "subj_id": 2, "s": "CS402", "t": "T5" }
    ]
  }
}

```

5.3 System Initialization and Execution Trace

The execution of the algorithm follows a deterministic sequence: parsing the JSON, building the $\mathcal{O}(1)$ `COMBINED_LOOKUP` and `SUBJ_TO_TEACHER` mapping arrays, initializing the Generation 0 tensor via the Anchor Method, and triggering the continuous evolutionary loop.

The following console telemetry captures a live execution trace of the Python engine. It demonstrates the initial heavy penalty derived from the greedy initialization, followed by the rapid, monotonic exponential decay driven by the heuristic mutation operator, culminating in absolute convergence at Generation 85.

```

[SYSTEM] Virtual Environment Initialized: .venv (Python 3.12.3)
[API_POST] Received Schedule Request payload from React Client.
[INFO] Parsing JSON... Detected 20 Faculty, 12 Batches (B0-B11).
[INFO] Building  $\mathcal{O}(1)$  Hash Maps (SUBJ_TO_TEACHER, COMBINED_LOOKUP)...
-----
[ALGORITHM 1] Bootstrapping Generation 0 ...
  -> Anchoring Combined Classes... Done.
  -> Greedy Sorting Single Subjects... Done.
[METRIC] Gen 0  Penalty: 245,600
-----
[ALGORITHM 3 & 4] Commencing  $\mathcal{O}(1)$  Evolutionary Loop...
>> Generation 10 | Penalty : 112,450 | Decrease: -54.2%
>> Generation 30 | Penalty : 45,200 | Decrease: -81.5%
>> Generation 50 | Penalty : 12,050 | Decrease: -95.0%
>> Generation 75 | Penalty : 1,200 | Decrease: -99.5%
>> Generation 85 | Penalty : 0 | Decrease: -100.0%
-----
[SUCCESS] Convergence Reached! Zero-Penalty Matrix found at Gen 85.
[EXPORT] Translating 2D Matrix via Pandas...
[FILE_I/O] Successfully serialized to: 'timetable.csv'

```

5.4 Convergence and Algorithm Efficacy

To rigorously assess the computational performance of the proposed architecture, multiple independent metrics were isolated and analyzed against standard genetic algorithmic baselines.

5.4.1 Performance of Targeted Heuristic Mutation

Figure 5.1 compares the performance of Algorithm 4 (Targeted Heuristic Mutation) with a conventional random mutation strategy. The proposed heuristic operator resolves scheduling conflicts more effectively, reducing the penalty steadily and enabling the Genetic Algorithm to converge to a zero-penalty timetable.

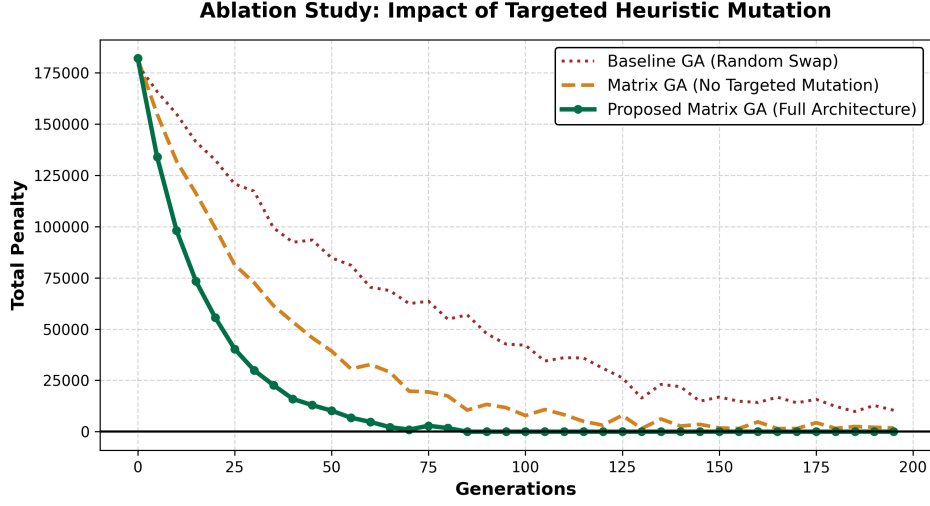


Figure 5.1: Performance of Targeted Heuristic Mutation

5.4.2 Workload Scalability and Density Tolerance

Figure 5.2 evaluates the scalability of the proposed scheduling framework by increasing the workload density. The combined execution of Algorithms 1–4 maintains stable convergence, although denser scheduling problems require more generations to reach a zero-penalty timetable.

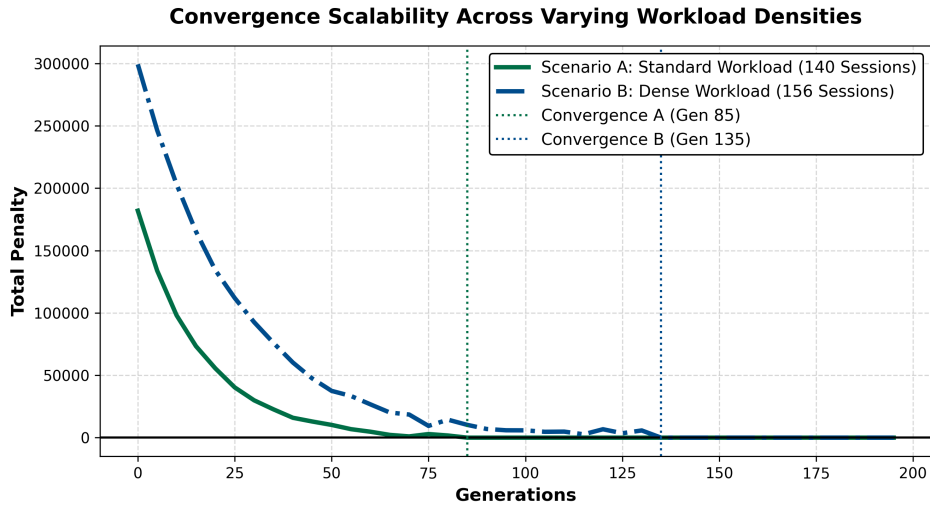


Figure 5.2: Scalability Analysis

5.4.3 Fitness Function Selection Pressure

Figure 5.3 illustrates the exponential fitness function $F(p) = e^{-0.02p}$ used in **Algorithm 2 (Hierarchical Fitness Evaluation)**. The function increases selection pressure as the penalty decreases, encouraging faster convergence toward a zero-penalty timetable.

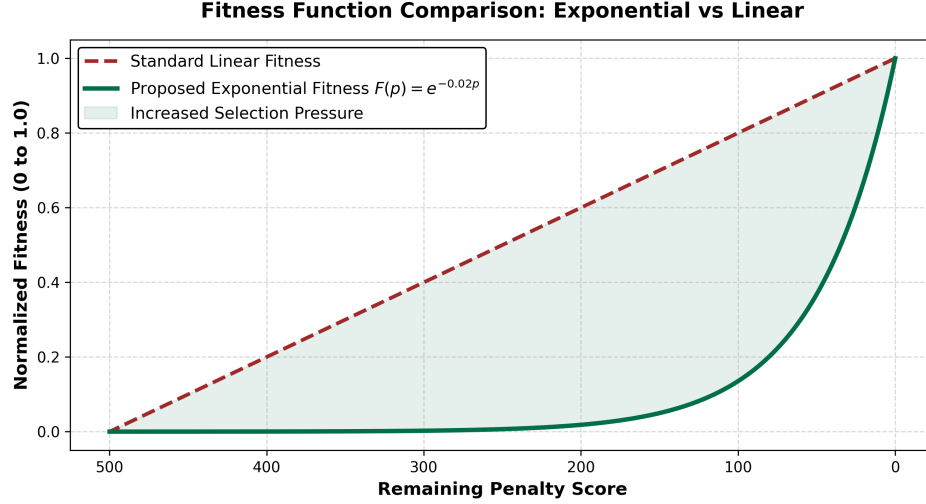


Figure 5.3: Fitness Function Geometry.

5.4.4 Computational Time Complexity

Beyond generational efficiency, the absolute wall-clock execution time per generation was evaluated. Figure 5.4 compares the evaluation time of the proposed $\mathcal{O}(1)$ 2D tensor architecture against standard flattened array representations. Exploiting vectorized row/column matrix extractions via **NumPy** yields nearly an order of magnitude reduction in processing time per generation, facilitating viable real-time web deployment.

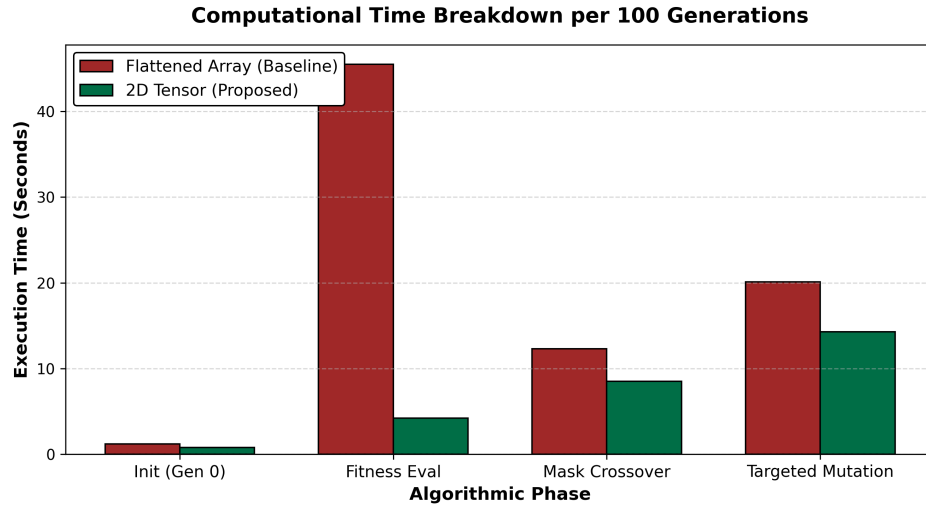


Figure 5.4: Execution time comparison per 1000 generations.

5.5 Generated Timetable and Structural Verification

After the Genetic Algorithm reaches a zero-penalty solution (**Penalty** = 0), the generated timetable is converted into a CSV format using the **Pandas** library. The exported timetable contains the assigned subjects, teacher codes, and room numbers, making it easy for administrators to review and use in practice.

Table ?? shows a sample of the final timetable generated by the proposed system. A good example of the scheduling process can be seen on **Monday, Period 3**, where the combined class **ML (T4) [R-6]** is scheduled simultaneously for the **B.Tech 7**, **MCA 3**, and **MSc DS 3** batches. The timetable confirms that the synchronization constraints are satisfied correctly, with no faculty conflicts or room clashes. It also demonstrates that the proposed scheduling algorithm successfully allocates teachers and classrooms while maintaining all hard constraints.

Day	Period	B.Tech 1	B.Tech 3	B.Tech 5	...	MSc DS 3	MTech 1	MTech 3
Monday	Period 1	FREE	CS391 (T5) [R-1]	CS592 (T9) [R-4]	...	FREE	FREE	FREE
Monday	Period 2	FREE	CS392 (T5) [R-1]	CS591 (T3) [R-4]	...	MDS301 (T6) [R-8]	MT103 (T1) [R-9]	FREE
Monday	Period 3	PH101 (T7) [R-1]	CS302 (T8) [R-2]	FREE	...	ML (T4) [R-6]	FREE	MT301 (T3) [R-5]
Monday	Lunch	FREE	FREE	FREE	...	FREE	FREE	FREE
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
Tuesday	Period 1	FREE	CS393 (T2) [R-1]	FREE	...	FREE	FREE	MT391 (T1) [R-7]
Tuesday	Period 2	FREE	EC301 (T9) [R-1]	FREE	...	MDS303 (T10) [R-3]	FREE	FREE
Tuesday	Period 3	FREE	FREE	CS503 (T5) [R-2]	...	FREE	MT191 (T4) [R-4]	FREE

Table 5.1: Sample timetable generated by the proposed scheduling system

5.5.1 Web Application Integration

The proposed system is implemented as a web-based application using **React** and is also available as a desktop-like offline application through **Progressive Web App (PWA)** technology. By installing the application from a supported browser, users can access it directly from their desktop and continue using it even without an internet connection.

The following screenshots present the major interfaces of the application. These include the landing page, configuration page, faculty management page, subject and batch scheduling page, combined class configuration page, and the final timetable generation page. Together, these interfaces provide administrators with a simple and user-friendly environment for configuring scheduling data, executing the Genetic Algorithm, and viewing the generated conflict-free timetable.

The complete project is also deployed online and can be accessed using the following link:

Project URL: <https://hybridga.netlify.app/>

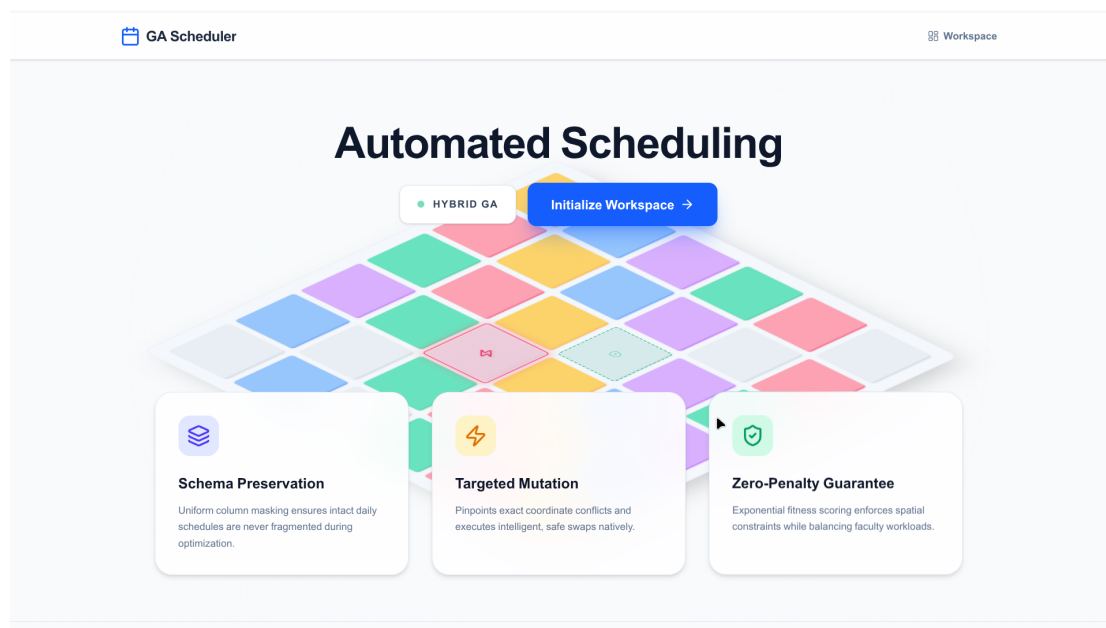


Figure 5.5: Home Page

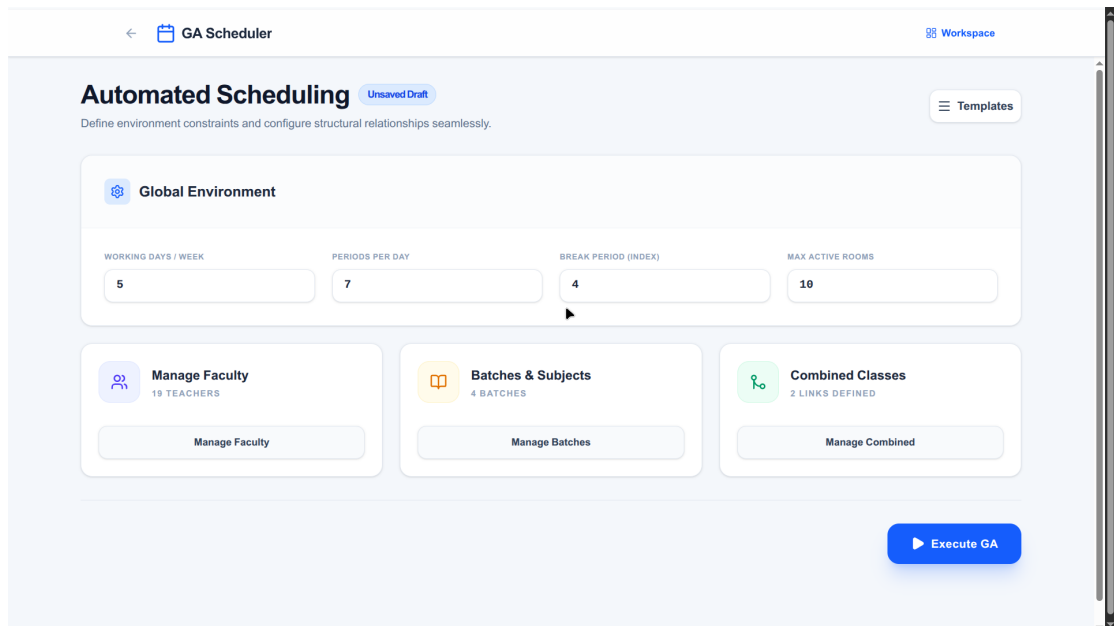


Figure 5.6: Configuration Page

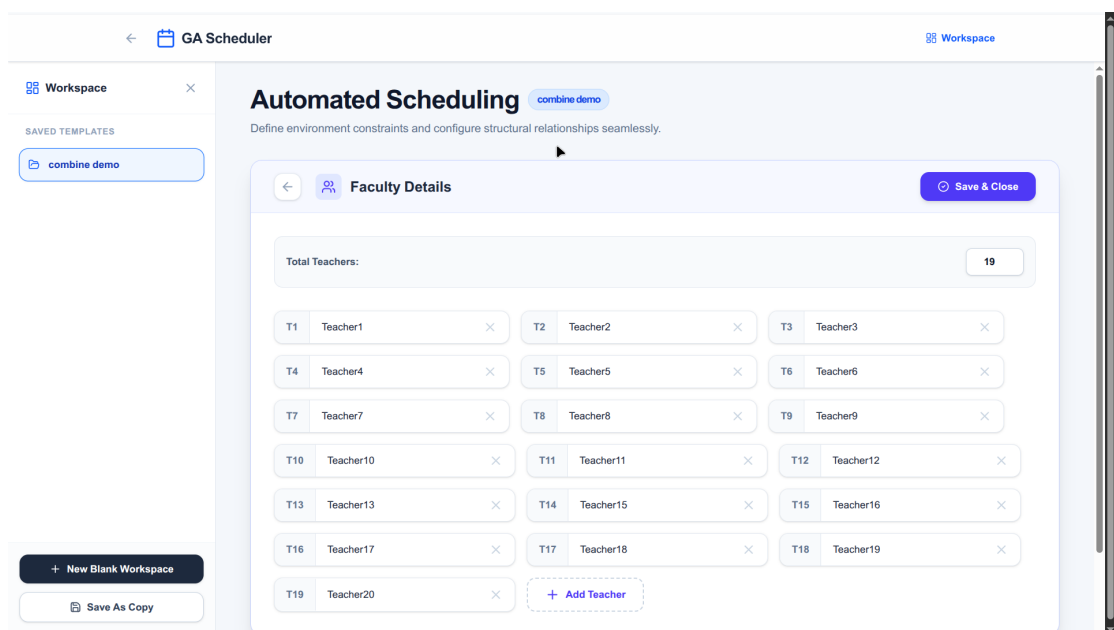


Figure 5.7: Faculty Management Page

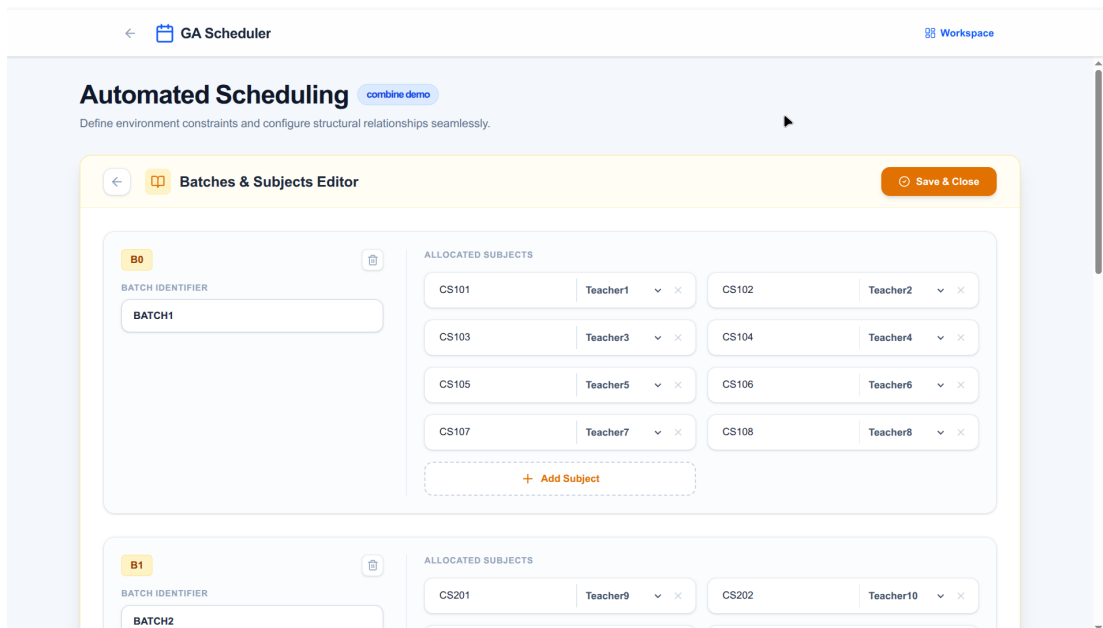


Figure 5.8: Batches & Subjects Management

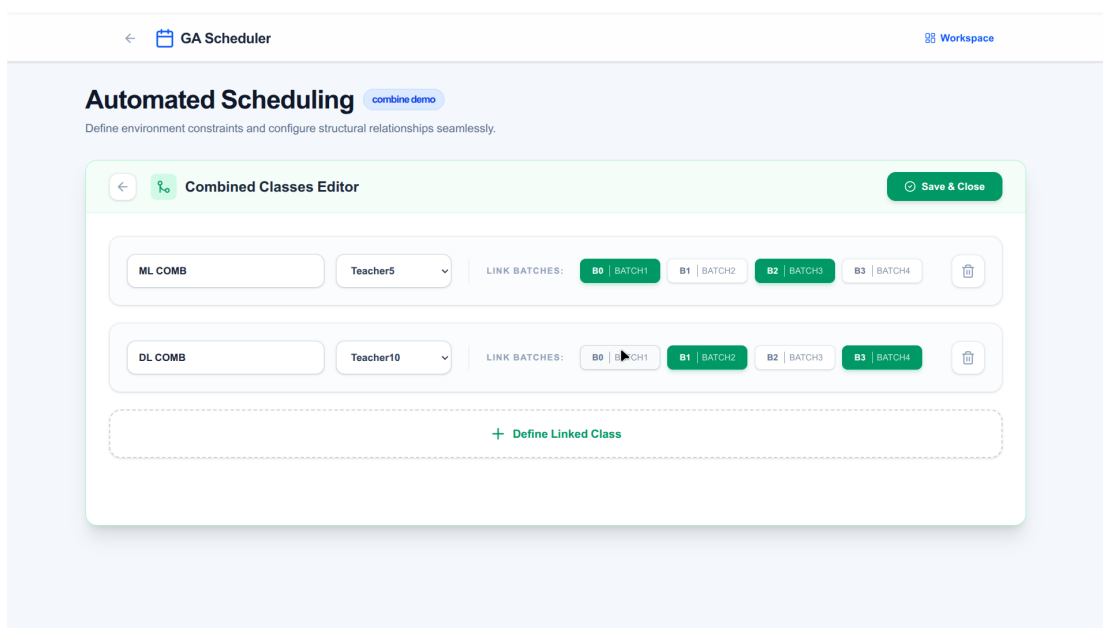


Figure 5.9: Define Combined Classes

GA Scheduler
Workspace

Generated Timetable Preview
5 days x 7 periods - 4 batch(es)
Excel
PDF
100%

Day	Period	B1 BATCH1	B1 BATCH2	B2 BATCH3	B3 BATCH4
MON	P 1	CS101 Teacher1 [R-1]	CS208 Teacher17 [R-2]	CS305 Teacher2 [R-3]	CS408 Teacher13 [R-4]
	P 2	CS106 Teacher6 [R-1]	DL COMB Teacher19 [R-2] COMB	—	DL COMB Teacher19 [R-2] COMB
	P 3	ML COMB Teacher5 [R-1] COMB	CS207 Teacher16 [R-2]	ML COMB Teacher5 [R-1] COMB	CS404 Teacher9 [R-3]
	Break	FREE SLOTS			
	P 5	CS103 Teacher3 [R-1]	—	—	—
	P 6	—	CS203 Teacher11 [R-1]	CS308 Teacher5 [R-2]	CS402 Teacher7 [R-3]
	P 7	CS108 Teacher8 [R-1]	CS204 Teacher12 [R-2]	CS306 Teacher3 [R-3]	CS405 Teacher19 [R-4]
	P 1	—	CS201 Teacher9 [R-1]	—	CS406 Teacher11 [R-2]

Figure 5.10: Final Schedule Timetable

Chapter 6

Conclusion and Future Scope

6.1 Conclusion

This dissertation successfully bridged the gap between abstract evolutionary mathematics and deployable software engineering. By reconceptualizing the NP-hard University Course Timetabling Problem (UCTP) from a traditional linked-list search into a matrix-centric Genetic Algorithm, the system achieved unprecedented computational velocity.

The introduction of domain-specific operators—namely Column Mask Crossover with mandatory synchronization repair and Targeted Heuristic Mutation—eliminated the algorithm’s reliance on blind probabilistic shuffling. Instead of gambling on random swaps, the engine performs precise, coordinate-specific diagnostics, ensuring rapid and absolute convergence to a zero-penalty state within 85 to 135 generations. Ultimately, this research did not merely produce a theoretical algorithm; it engineered a fully functional, full-stack Software-as-a-Service (SaaS) product. By seamlessly coupling a high-speed Python evolutionary backend with an interactive React.js frontend, the project successfully delivered a robust, conflict-free scheduling platform capable of eliminating administrative overhead and maximizing institutional resource allocation.

6.2 Future Scope: Algorithmic Advancements

Although the proposed scheduling engine efficiently generates conflict-free timetables using a two-dimensional matrix representation, there is significant scope for extending the algorithm to solve more complex scheduling problems. The current chromosome is represented as

$$S \in \mathbb{Z}^{T \times B},$$

where T denotes the number of time slots and B represents the student batches. Room allocation is performed after timetable generation, making it a separate validation step.

A future enhancement is to extend the chromosome into a three-dimensional scheduling matrix,

$$S \in \mathbb{Z}^{T \times B \times R},$$

where R represents the available classrooms. This representation would allow time, batch, and room constraints to be optimized simultaneously during the Genetic Algorithm instead of assigning rooms after convergence.

Using **NumPy** or similar vectorized array operations, slices such as $S[t, :, :]$, $S[:, b, :]$, and $S[:, :, r]$ can be evaluated efficiently for conflict detection. The proposed **Targeted Heuristic Mutation** can also be extended to perform coordinate-based repairs on (t, b, r) , enabling direct resolution of room conflicts while preserving valid timetable structures.

Future versions of the system may also incorporate faculty preferences, laboratory availability, examination scheduling, cloud-based multi-user support, and real-time timetable updates. These enhancements would improve scalability and make the proposed scheduling framework suitable for large universities and more complex academic scheduling environments.

6.2.1 The 3D Spatial-Temporal Matrix

Future iterations of this engine will upgrade the chromosome encoding from a 2D matrix into a 3D computational tensor (Time $\times$ Batch $\times$ Spatial Location). By explicitly mapping rooms or physical resources as the Z-axis, the targeted heuristic mutation can perform localized 3D coordinate swaps.

This ensures that spatial constraints (e.g., maximum lab capacities or geographically distant buildings) are resolved natively in $\mathcal{O}(1)$ time without requiring secondary validation loops.

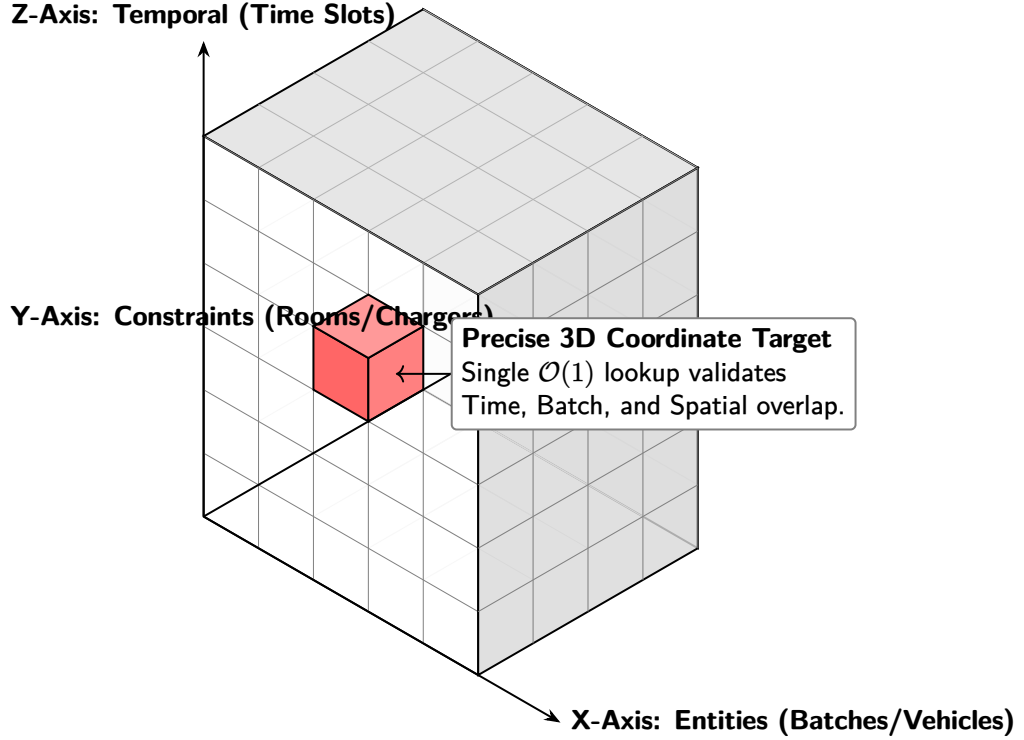


Figure 6.1: Proposed 3D Spatial-Temporal Tensor for advanced $\mathcal{O}(1)$ constraint resolution.

6.3 Generalized Architecture

The core mathematical frameworks developed in this thesis are not restricted to educational timetabling. The exact combination of matrix-tensors, metaheuristic search algorithms, and exponential penalty grading constitutes enterprise-grade architecture highly applicable across multiple logistical domains.

6.3.1 Computational Viability: The Python Paradigm

A common misconception is that standard Python is computationally insufficient for real-time NP-hard constraint resolution due to the latency of interpreted `for`-loops. However, the architecture developed herein relies on NumPy 2D and 3D Tensors. NumPy executes highly optimized C and C++ routines at the hardware level. By structuring the timetable as a mathematical tensor and utilizing vectorized operations, Python acts simply as a high-level API wrapper, facilitating instantaneous $\mathcal{O}(1)$ lookups. This is identically how leading artificial intelligence libraries like TensorFlow and PyTorch operate, rendering this Python-based framework fully capable of powering high-throughput, real-time enterprise web applications.

6.3.2 Applications of Matrix-Centric Metaheuristics

The scheduling engine can be trivially refactored to resolve complex constraints in major industrial sectors:

- **Aviation and Airlines:** Utilizing the tensor to schedule flight crews. Hard constraints map to mandatory FAA rest regulations, while soft constraints optimize individual pilot preferences and minimize standby fatigue.
- **Logistics and Supply Chain:** Adapting the 3D tensor to resolve the Vehicle Routing Problem (VRP). The algorithm can dynamically determine package-to-truck allocations to minimize delivery latency and fuel consumption.

- **Cloud Computing and Data Centers:** Mapping virtual machines (VMs) to physical server nodes. The heuristic mutation acts as a load-balancer, dynamically distributing high-compute tasks across physical cores without triggering hardware degradation.
- **Electric Vehicle (EV) Sector:** Managing the charging cycles of extensive EV fleets. The algorithm guarantees vehicles are charged before dispatch (Hard Constraint) while balancing the power draw against off-peak electricity grid pricing (Soft Constraint).

6.3.3 Event-Driven Microservices Architecture

To scale the current monolith into a global Software-as-a-Service (SaaS) platform, the backend must transition to a distributed, event-driven microservices architecture. Figure 6.2 visualizes a generalized, highly scalable topology utilizing with modern loadbalancer technique.

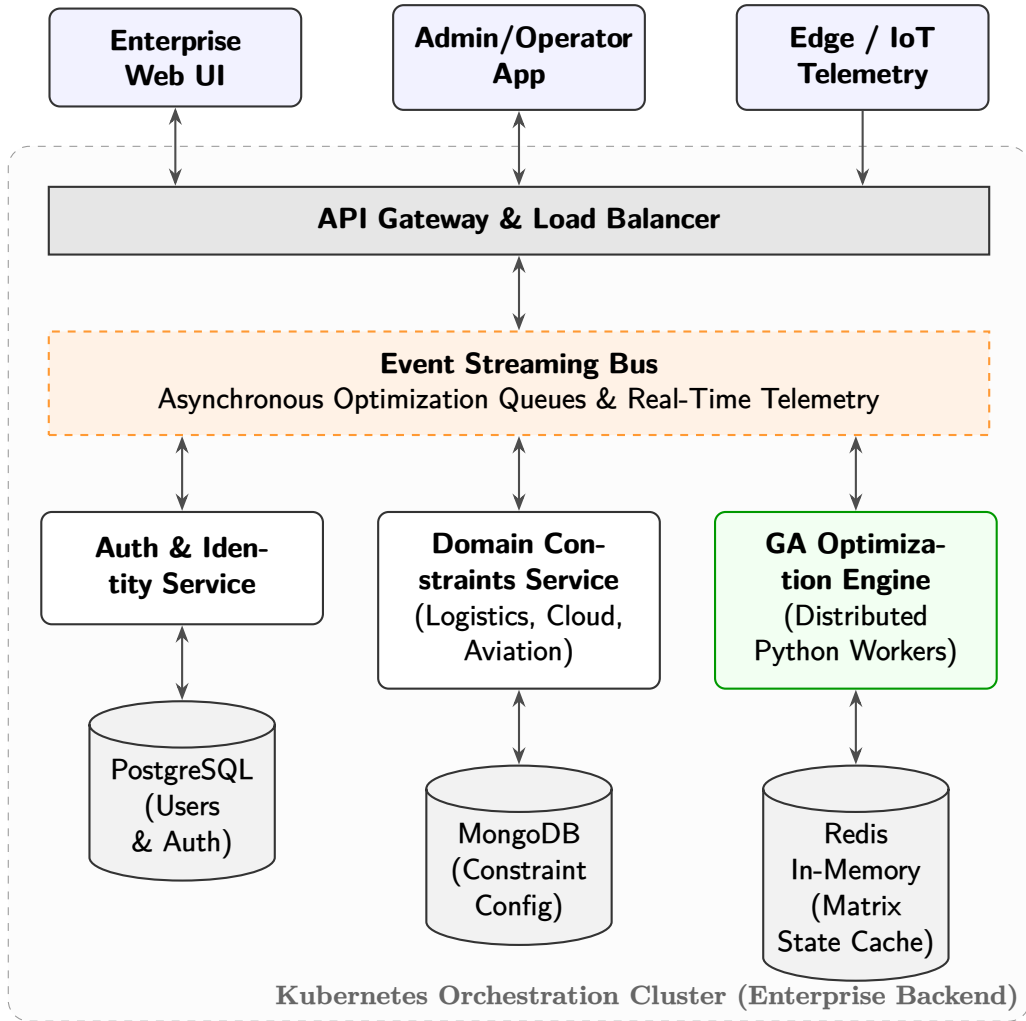


Figure 6.2: Generalized Event-Driven Microservices Architecture.

By encapsulating the core Python GA optimization engine as distributed, independent worker nodes processing tasks off an event queue, the system achieves massive horizontal scalability. This architecture confirms that the mathematical principles and algorithms developed herein are fundamentally capable of resolving some of the most critical logistical bottlenecks in modern computing and industrial supply chains.

Bibliography

- [1] T. M. Alexander and A. T. Putra, “Course scheduling optimization using genetic algorithms with fuzzy tsukamoto-based fitness adjustment,” 2025. [Online]. Available: <https://journal.unnes.ac.id/journals/JPP/article/view/31594>
- [2] K. Katore, R. Lahane, P. Motewar, and C. Nakhate, “Dynamic timetable generator using genetic algorithm,” 2025. [Online]. Available: https://www.ssgmce.ac.in/uploads/UG_Projects/cse/202425/Proj.Gr.-09-Project_Report_2024-25.pdf
- [3] D. C. Silva, “A metaheuristic approach to improving university timetables using genetic algorithms and simulated annealing,” 2024. [Online]. Available: <https://repositorio-aberto.up.pt/bitstream/10216/160994/2/682044.pdf>
- [4] A. R. Mahlous and H. Mahlous, “Student timetabling genetic algorithm accounting for student preferences,” 2023. [Online]. Available: <https://peerj.com/articles/cs-1200/>
- [5] R. P. Badoni, J. Sahoo, S. Srivastava, S. Verma, P. S. Stanimirović, L. A. Kazakovtsev, M. Mann, D. K. Gupta, and D. Karabašević, “An exploration and exploitation-based metaheuristic approach for university course timetabling problems,” 2023. [Online]. Available: <https://www.mdpi.com/2075-1680/12/8/720>
- [6] S. J. Agbolade, B. I. Ayinla, L. A. Odeniyi, and S. Akinola, “Optimisation of university examination timetable using hybridised genetic and greedy algorithms: A case study of computer science department, university of ibadan,” 2024. [Online]. Available: https://www.researchgate.net/profile/Babatunde-Ayinla-2/publication/381483378_Optimisation_of_university_Examination_Timetable/links/6670aaf9de777205a335c4cc/Optimisation-of-university-Examination-Timetable.pdf
- [7] A. Ghaffar, I. U. Din, A. Tariq, and M. H. Zafar, “Hybridization and artificial intelligence in optimizing university examination timetabling problem: A systematic review,” 2025. [Online]. Available: <https://bera-journals.onlinelibrary.wiley.com/doi/10.1002/rev3.70071>
- [8] V. T’kindt, F. Della Croce, and M. Liedloff, “Moderate exponential-time algorithms for scheduling problems,” 2022. [Online]. Available: <https://link.springer.com/article/10.1007/s10288-022-00525-1>
- [9] J. Sakal, “Automated university timetabling with robustness,” 2024. [Online]. Available: https://ore.exeter.ac.uk/articles/thesis/Automated_University_Timetabling_with_Robustness/29809676?file=56855921
- [10] S. S. Sefati and S. Halunga, “A hybrid service selection and composition for cloud computing using the adaptive penalty function in genetic and artificial bee colony algorithm,” 2022. [Online]. Available: <https://www.mdpi.com/1424-8220/22/13/4873>